

Neutrosophic-based domain-specific languages and rules engine to ensure data sovereignty and consensus achievement in microservices architecture

2

Haitham A. El-Ghareeb

Information Systems Department, Faculty of Computers and Information Sciences,
Mansoura University, Mansoura, Egypt

2.1 Introduction

A monolithic application is an application with a single large codebase/repository that offers tens or hundreds of services using different interfaces such as HTML pages, web services, and/or REST services [1]. A monolithic application has most of its functionality within a single process that is componentized with internal layers or libraries [2]. Monolithic applications scale out by cloning the application on multiple servers or multiple virtual machines. Fig. 2.1 presents a typical monolithic application structure and scaling mechanism. Monolithic applications rely heavily on layered architectures (as depicted in Fig. 2.1 per application within a physical or virtual server). Each box in Fig. 2.1 presents a running instance of the application on a separate server.

Microservices architecture has developed from a promising architecture into a de facto standard in designing and developing distributed applications. Microservices architecture is used by large companies like Amazon, Netflix, and LinkedIn to deploy large applications in the cloud as a set of small services that can be developed, tested, deployed, scaled, operated, and upgraded independently, allowing these companies to gain agility, reduce complexity, and scale their applications in the cloud in a more efficient way [1].

2.2 Microservices

Microservices can be thought of as an architectural style for developing applications as suites of small and independent (micro)services. Each Microservice is built around a business capability; it runs in its own process and it communicates with the other Microservices in an application through lightweight mechanisms (e.g., HTTP Application Programming Interfaces (APIs)) [3].

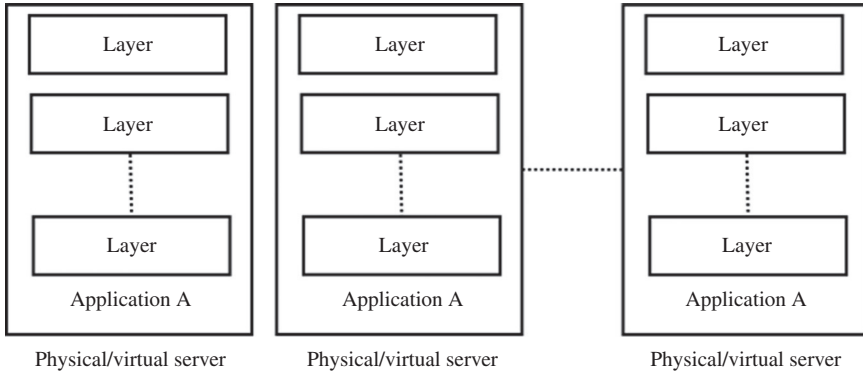


Fig. 2.1 Monolithic application structure and scaling mechanism.

A Microservice is either a service-oriented architecture (SOA), if done right, or it is a totally new architecture. Microservices have learned from SOA mistakes and failures [4–6]. One important rule for Microservices is data sovereignty.

2.2.1 Data sovereignty

Each Microservice must hold and control its data. No Microservice should be able to access another Microservice’s data. Integration at the database layer is not recommended, because it introduces coupling between the data representation internally used by multiple Microservices. Instead, Microservices should interact only through well-defined APIs, to provide a clear mechanism for managing the state of the resources exposed by each Microservice.

2.2.2 Integrated data

Microservices architecture-based solutions do not act in isolation. There must be relationships between different Microservices. Relationships between related entities can be implemented using hypermedia, so that representations retrieved from one Microservice API can include links to other entities found on other Microservice APIs [7]. Such a solution faces many of challenges, and can easily become unreliable. Broken links on the World Wide Web are a live example of such a challenge.

2.2.3 Statefulness

A stateful system produces outputs that depend on the state generated in previous interactions. An e-Commerce application case study will be presented in [Section 2.7](#) to depict this idea. e-Commerce is stateful if it remembers previous shopping activities and past visits to the online shop. Such activities are important for personalization, recommendation, and security.

2.2.3.1 *Stateless microservices*

In many cases, stateless Microservices are ideal because they exploit the benefits of cloud computing, such as on-demand elasticity, load balancing, high availability, and high reliability through redundancy [8].

2.2.3.2 *Stateful microservices*

Stateful Microservices require a more complex logic for managing the state, are less scalable, and do not facilitate high availability and high reliability. Hot standby is a redundant method in which one system runs simultaneously with an identical primary system. Upon failure of the primary system, the hot standby system immediately takes over, replacing the primary system. Fail-over scenario is switching to a redundant or standby computer server, system, hardware component or network upon the failure or abnormal termination of the previously active application, server, system, hardware component, or network. State information has to be made persistent and synchronized in hot standby configurations and recreated in fail-over scenarios. Even without failure, statefulness leads to session affinity, which can decrease throughput and increase latency due to the need to wait for specific stateful instances [8].

Like any other architectural style, Microservices have easy parts and tough parts. Tough parts always define success from failure. Easy parts in Microservices architecture include communication between different Microservices. It is widely accepted that the REST protocol is the de facto one for communication. Restful APIs solved a big problem in Microservices architecture, mainly in standardizing communication. Stateless Microservices facilitate autonomy and isolation, so we guarantee that we acquire all Microservices benefits and promises. It has become widely accepted that we tend to make Microservices as stateless as possible.

Stateless Microservices are perfect. However, applications are not stateless. Microservices need to reflect applications' statefulness nature, and hence comes the challenge.

2.2.4 *Containerized applications using Docker*

2.2.4.1 *Containers*

Container technology is having a profound impact on distributed systems and cloud computing. While virtual machines provide an abstraction at the hardware level, the container model virtualizes operating system calls so that, typically, many containers share the same kernel instance [9]. Containers are like a very lightweight virtual machine. Virtual machines are a fairly large-weight computer resource. An average virtual machine is a copy of an operating system running on top of a hypervisor running on top of physical hardware, which an application is then on top of. That presents some challenges for speed and performance [10].

2.2.4.2 Docker

This is a disruptive technology which changes the way applications are being developed and distributed. With a lot of advantages, a Docker is a very good fit to implementing Microservices architecture [11]. A Docker is a container virtualization technology that attempts to overcome virtual machine's challenges.

2.2.4.3 Docker Swarm

The Docker has recently introduced its distributed system development tool called Swarm, which extends the Docker container-based software development process on multiple hosts in multiple clouds without any interoperability issues. Docker Swarm-based distributed software development is a new approach for the cloud industry; nonetheless, it has huge potential to provide a multicloud development environment without increasing the complexity of it [12].

2.3 Domain-specific languages

A domain-specific language (DSL) is a programming language that is specialized to a particular application domain [13]. Although DSL advantages have been known for a long time, they have not achieved widespread adoption yet [14]. Languages that were created for particular domains include FORTRAN used to allow direct mathematical formula, Structured Query Language for database access and manipulation, and Algol for algorithm specification [15].

2.3.1 DSL components

Programs that recognize languages are called parsers or syntax analyzers [16]. Syntax refers to the rules governing language membership. Grammar is just a set of rules, each one expressing the structure of a phrase.

Lexer. The program that tokenizes the input is a lexer. It is also known as a tokenizer. It is the first step toward a parser. The process of grouping characters into words or symbols (tokens) is called lexical analysis or simply tokenizing [16].

Parser. The second stage is the actual parser and feeds off of these tokens to recognize the sentence structure. Most parsers build a data structure called a parse tree or syntax tree that records how the parser recognized the structure of the input sentence and its component phrases [16].

2.4 Rules engine

Business rules are full of changes in the business world. Businesses need to adapt to changeable business rules. A business rules engine is a software system that executes one or more business rules in a runtime production environment. The rules might come from legal regulation ("An employee can be fired for any reason or no reason

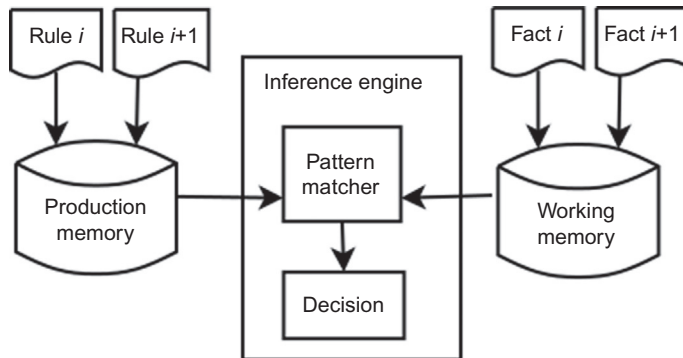


Fig. 2.2 Rules engine components.

but not for an illegal reason”), company policy (“All customers who spend more than \$100 at one time will receive a 10% discount”), or other sources. Rule engine software is commonly provided as a component of a business rule management system which, among other functions, provides the ability to: register, define, classify, and manage all the rules, verify consistency of rules definitions, define the relationships between different rules, and relate some of these rules to IT applications that are affected or need to enforce one or more of the rules [17]. Fig. 2.2 depicts the most important rules engine components.

Rule-based systems have become increasingly popular due to three factors [18]:

1. better separation of concerns between the knowledge and its implementation logic in contrast to a hard-coded approach;
2. rule repositories that increase the visibility and readability of the knowledge; and
3. graphical user interfaces that render rules more usable while bridging the gap between users (e.g., domain experts) and IT specialists.

Drools is a hybrid chaining engine, meaning it can react to changes in data and also provides advanced query capabilities. Drools provides built-in temporal reasoning for complex event processing and is fully integrated with the jBPM project for BPMN2-based workflow [19].

The rest of this chapter is arranged as follows: Section 2.5 introduces the basic and required concepts of neutrosophic sets and theory. Section 2.6 presents the problem definition that is tackled for a solution in this chapter. Section 2.7 presents the problem definition in a concrete description. Section 2.8 presents a case study adopting DSL and highlighting how it can be utilized in solving different challenges. Section 2.9 presents the conclusion and future work. The chapter concludes with references.

2.5 Neutrosophic theory

Neutrosophic sets have been introduced to the literature by Smarandache to handle incomplete, indeterminate, and inconsistent information [20]. In neutrosophic sets, indeterminacy is quantified explicitly through a new parameter I . Truth-membership

(T), indeterminacy membership (I), and falsity-membership (F) are three independent parameters that are used to define a neutrosophic number.

$$\dot{N} = \left\{ \left\langle x; T_{\dot{N}}(x), I_{\dot{N}}(x), F_{\dot{N}}(x) \right\rangle, x \in X \right\} \quad (2.1)$$

$$x \in X, \quad T_{\dot{N}}(x), I_{\dot{N}}(x), F_{\dot{N}}(x) \in [0, 1] \quad (2.2)$$

2.5.1 Single-valued neutrosophic number

This presents the basic element and the most widely used neutrosophic number. Single-valued neutrosophic numbers' (SVNNs) operations and their implementation are presented in this section.

Deneutrosophy is the process where neutrosophic scales/numbers are converted to crisp values by applying score functions of $s(a_{ij})$ as illustrated in Eq. (2.3) [22].

$$s(a_{ij}) = \left| \left(l_{rij} \times m_{rij} \times u_{rij} \right)^{\frac{T_{rij} + I_{rij} + F_{rij}}{9}} \right| \quad (2.3)$$

2.5.2 Single-valued neutrosophic set

A single-valued neutrosophic set (SVNS) consists of multiple SVNNs. Aggregation operations are presented in this section. Implementation details simplified the calculation, and hopefully will act as an enabler for researchers in academia and developers in industry as a guidance and concrete implementation on how to adopt neutrosophic sets in real-world applications.

2.6 Problem definition

A Microservices architecture approach introduces a lot of new complexity and requires in application developers a certain level of maturity in order to apply the architectural style confidently [11]. Let us take a closer look at some Microservice design architecture alternatives.

Microservices mesh architecture. Fig. 2.3 depicts a Microservices architecture, in which each Microservice is autonomous and satisfies data sovereignty rules. This figure presents the communication method between different Microservices, and between different Microservices and front-end applications. They are connected using mesh topology. Full mesh topologies have the advantages of high fault tolerance. For this Microservices setup, an advanced front-end application based on a modern technology like Angular, React, Vue, or others can make use of component-based front-end applications. Each component can get its own data from a different Microservice. However, it is clear that this architectural setup suffers in scalability. Full mesh

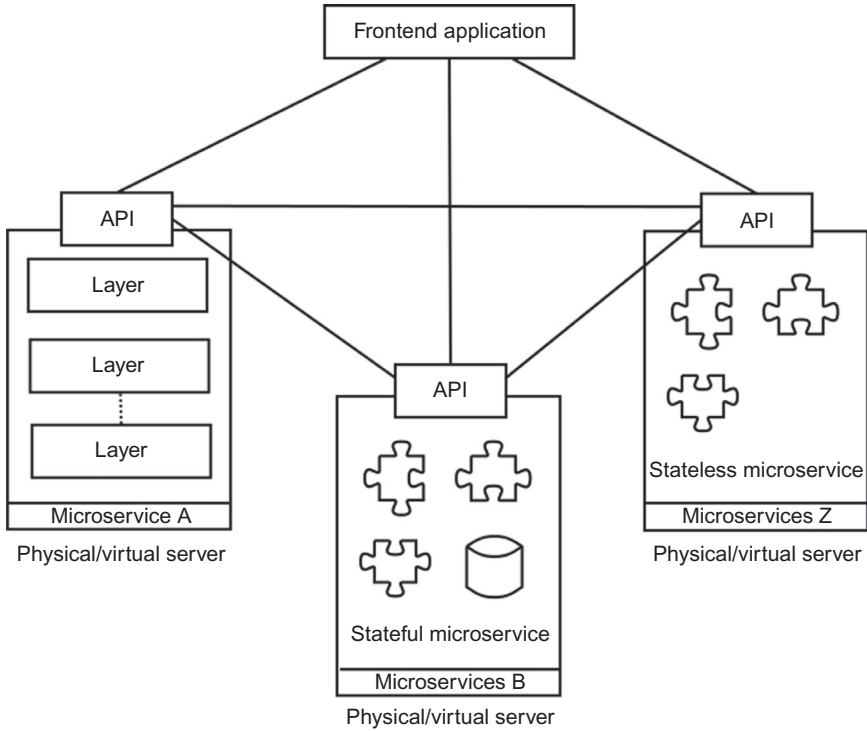


Fig. 2.3 Microservices mesh architecture structure.

topology requires tremendous connections, and they are tough to manage, monitor, and scale. Eq. (2.4) represents the total number of connections required for full mesh topology. Besides, this architecture setup suffers from a severe disadvantage: a global application state. In this Microservices architecture setup, each Microservice has its own state. What if our application needs to take decisions based on a global state, that is, a state from different Microservices? What about integrated data from different Microservices? It is not applicable and not accepted to leave such a decision for the front-end application.

$$T_c = \frac{n*(n-1)}{2} \quad \text{where } n = \text{no. of nodes} \quad (2.4)$$

Microservices mesh architecture with gateway. Fig. 2.4 presents an adaptation to Fig. 2.3 that adds an API Gateway to the system. This architecture solves the need for the polynomial number of connections between Microservices, and presents a centralized method to manage the global system state. However, presenting a central point is a clear contradiction with Microservices architectural objectives. This is a reproduction of SOA choreography and orchestration: Two services-based design

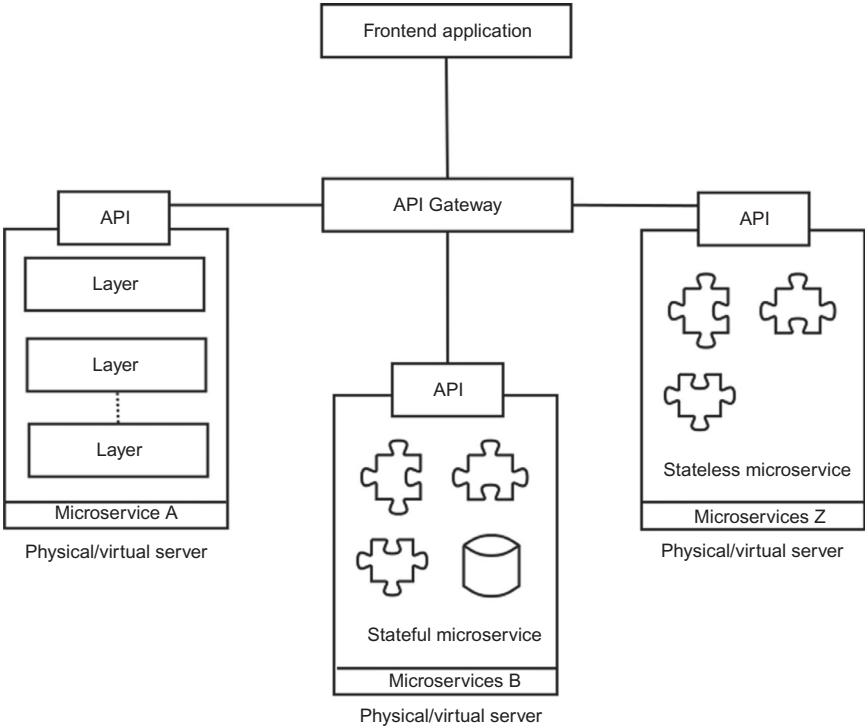


Fig. 2.4 Microservices mesh architecture structure with gateway.

patterns that were not successful within SOA, besides of course, the central point of failure, scalability, and availability bottleneck that this solution suffers from.

Microservices mesh architecture with Swarm manager and event bus. Fig. 2.5 presents a midway solution between the previous two Microservices architectures that attempts to make use of a simple event bus that supports publish/subscribe events management, so we get rid of full mesh topology, and relies on Swarm/cluster management for monitoring, tracking, load balancing, and directing requests and messages among Microservices.

Certainly there are different Microservices architectural setups, each with its own advantages and challenges. Picking the most suitable one relies mainly on the problem in hand. Attempting to present highly modular, easily scalable, enterprise-wide applications, chapter focus will be on Microservices mesh architecture structure with Swarm manager and event bus. This architecture, though it is the most sophisticated among the presented ones, is the one that makes the most benefit from different cloud computing providers' services (benefiting from the Swarm manager). However, it suffers from different challenges. Those challenges can be generalized to different Microservice architectures. In this chapter, we attempt to solve the following challenges:

1. What happens if the service provided to the user needs integration of data between different Microservices?

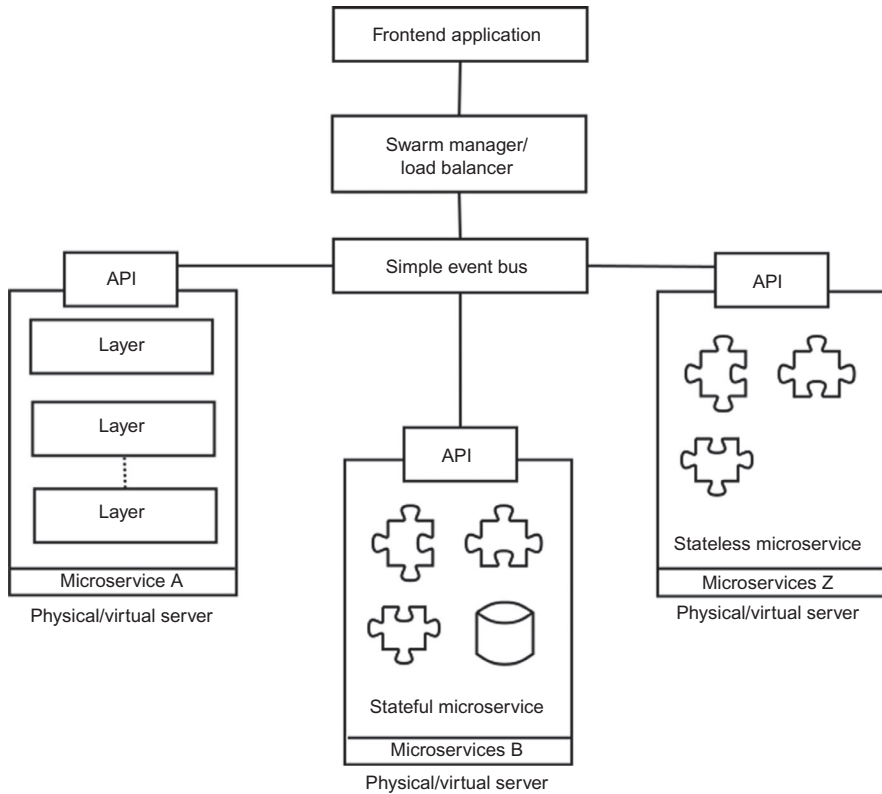


Fig. 2.5 Microservices mesh architecture structure with Swarm manager and event bus.

2. How can we handle the case that data will be calculated depending on values from other Microservices?
3. What happens if one of those Microservices fails?
4. What are the procedures to be followed to guarantee accepted levels of scalability and availability?
5. Applications must have the ability to present clones of the current Microservices as needed. What are the means to do so?
6. What about databases for those newly generated Microservices?

We can rephrase the previous challenges in simple words: We need to manage the global application state, on both infrastructure and database levels. However, Microservices-based applications now consist of autonomous Microservices, that are independent, and do not rely on each other. Each Microservice has its own data, and they must satisfy data sovereignty rules.

We need to find a suitable solution that satisfies the following conditions:

1. The proposed solution does not rely on centralized or shared databases.
2. The proposed solution must be as flexible and adaptable as possible.
3. The Microservices must adhere to different changing business rules, especially for global software solutions.

2.7 Case study problem definition: e-Commerce

This section presents a concrete description of the problems presented in [Section 2.6](#). The classical attempt to design and implement a Microservices-based e-Commerce includes some activities as follows. e-Commerce is divided into basic Microservices, using Domain-Driven Design, into:

1. products Microservice;
2. orders Microservice;
3. users Microservice; and
4. front-end Microservice.

Such a design can be containerized into different Microservices easily. Taking the easy way in designing and implementing an e-Commerce Microservices solution will guide us to the following steps:

1. identify the different Microservices;
2. design, develop, and deploy those Microservices (most probably as containers);
3. Microservices' main communication method would be the REST Protocol through Restful APIs;
4. deploy those Microservices into Swarm to provide scalability, monitor, and manage different services to detect failures, ease scalability, and benefit from other Swarm capabilities; and
5. develop the front-end application, which communicates with different Microservices via the Swarm manager(s).

Hence come the main challenges. Those challenges are best depicted through different scenarios, as follows:

1. *Scenario 1:* The user is viewing the catalog items. It has become widely accepted that users do not have to know the item price displayed to other users, because our e-Commerce needs to provide different prices for different items to different users based on different criteria. Criteria might include:
 - purchase history;
 - user loyalty; and
 - personalized parameters, like birthdays, geolocation, personal interests, etc.
 So, our e-Commerce needs certain information about the user from other Microservices (mainly user Microservice in this scenario) to be able to respond to certain queries. In addition, those rules must not be hard-coded in the Microservice, to be able to change them easily in the future, and provide high levels of adaptability and configurability. Hard-coding such rules will lead to application redeployment to change such rules.
2. *Scenario 2:* The user has placed a new order; however, this order is very suspicious. Order characteristics include:
 - The order is originating from an IP address that the user has never used before.
 - This IP address belongs to a country/region that the user was never recorded as having logged in from before.
 - The order includes a new product/category that the user has not ordered before.
 - The shipping address is completely a new one.
 - The shipping address belongs to an address that is in a country that was not logged for this user before.

Such a scenario might or might not be natural; however, it is certainly suspicious. To resolve this issue, e-Commerce solution must be able to utilize integrated data from different Microservices, and to check certain rules. Those rules need to be changed based on current security threats, and depending on many criteria.

3. *Scenario 3*: It is White Friday, and suddenly we have a real traffic overload. Our e-Commerce needs to make new sets of different Microservices. As stated earlier, Microservices can be divided into stateless and stateful Microservices. Replicating stateless Microservices is easy and efficient. However, replicating stateful Microservices is a challenging task.
 - Containerized applications rely on building and creating containers from images. Images are like blueprints in object-oriented programming, and containers are more like objects.
 - Every newly created container is a representation of the image state. It is completely in an initial state.
 - So, if our e-Commerce application relies on creating new containers from images, the newly created containers will fall behind current executing containers.
 - A proposed solution might be to make clones of current executing containers.
 - To make a clone of the current executing containers, we have to:
 - stop the current container, which will affect overall system availability; and
 - find a solution to manage the nonconsistent state duplication, for example, if the user is in the middle of placing an order, and we replicated the container, this means we have two running Microservices instances (containers) with an incomplete order, and the user will continue with one of them only. So, what about the status of the newly cloned container?

2.8 Case study solution

2.8.1 DSL

In this chapter, we present neutrosophic-based DSL and rules engine as a solution to the above-mentioned problems. DSLs have existed for a long time. It is time to re-evaluate their suitability to tackle tough challenges in Microservices. [Listing 2.1](#) presents an event of type Purchase Operation Pending with Global ID abc123xyz that took place on Node ID 1234 with items on the cart right then. There are some notes on this grammar:

Listing 2.1 e-Commerce in JSON format

```
{
  "guid" : "abc123xyz",
  "description" : "Purchase Operation Pending",
  "cartItems" : { "book": "Microservices and DSLs", "chapter":
    "Enterprise Integration" },
  "nodeID" : "1234"
}
```

- The JSON format has been chosen as e-Commerce DSL representation, so it can be processed later, even without the need of the proposed parser. JSON has native support in most modern programming languages.
- The proposed DSL grammar is not exclusive or complete. This is a proof of work that can be easily expanded and built upon.

The main contribution of this chapter is: our e-Commerce application uses the Microservices logs in an innovative manner. Logs here are not just entries that are used to record important events, so administrators can come back later to check what went wrong, or identify certain performance issues. Logs have become a DSL that can be parsed based on a syntax tree, like the one presented in Fig. 2.6 generated from Listing 2.1. Listeners and Visitors can later be generated for each node in this parse tree. Code can be executed based on the visited node, giving large flexibility and enabling an efficient rule-based engine among Microservices. Generated Listeners and Visitors form the basis of the Rules Engine that will direct the system’s decision based on the different inputs.

Treating log events as DSL helped solve e-Commerce challenges in different ways:

- DSL is programming language independent; it is more a grammar. DSL can be understood by different programming languages, so each Microservice can have its own autonomy, and can utilize different technology.
- DSL can specify the rules for the system to follow globally. DSL can easily act as an enabler for rules engines. Our e-Commerce Microservices can adjust their behavior based on DSL logs that are exchanged and delivered.
- Those rules include specifications about promotions, special discounts, special pricing, and security, among other specifications. Those rules are enough to solve the majority of issues discussed in the above-mentioned scenarios.

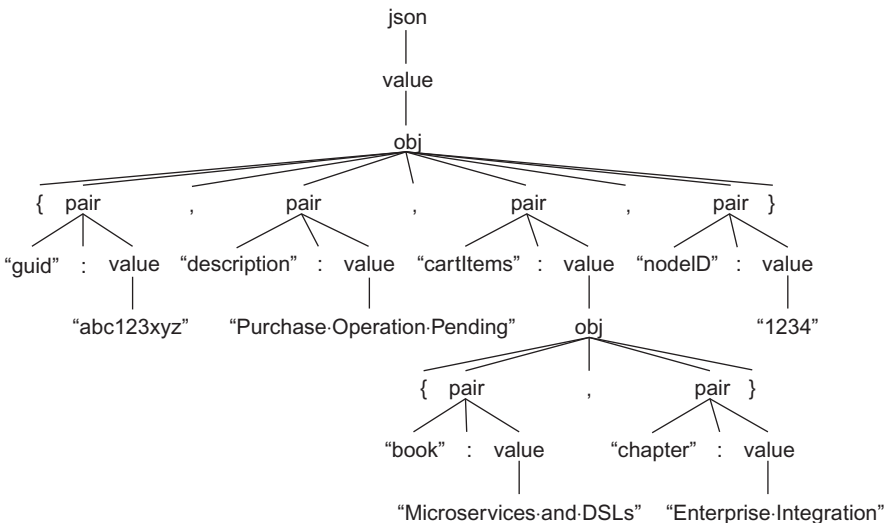


Fig. 2.6 Original DSL parse tree.

- For newly created Microservice container instances, those logs can be used to move the newly created Microservice state forward, so it overcomes the challenge of cloning the state from similar Microservices.
- Those events can be easily used to build event-dependency graphs. Such graphs are very useful and helpful for identifying conflicts, defining deadlocks, helping to solve eventual consistency problems. Such challenges are very well known in distributed systems, and they need communication and collaboration between different Microservices.

2.8.2 *Neutrosophic DSL*

Section 2.8.1 presented the simplest and most straightforward case, but it was necessary to present it this way to make the problem scenario and solution workflow clear. However, the scenario presented at Section 2.8.1 faces some challenges. Section 2.8.1 assumed that the user is dealing through the purchase process with only one Microservice, something that is not true. In distributed systems, it is well known that the system must keep a global state, so the user can continue processing from any Microservice—of course that of the same type of the user’s request. That means all running Microservices must negotiate a global state and reach consensus. Caching is a highly utilized technique to enhance distributed web applications. There are different caching algorithms and implementations depending on the case at hand; however, the simplest case involves that:

- if the user requests an item that is not in the cache, the Microservice reads it and keeps it in cache; and
- if the user updates an item, the Microservice updates its value at the cache first, then persists that update to the database later.

Caching provides many advantages to systems; however, it presents the problems of Dirty Reads and Dirty Writes. Simply, reading a value from a cache that was never persisted in the database, and executing further operations yields a wrong value. Another version of the problem is that writing a value to the cache that was never persisted leads to the loss of that value. There are different proposed solutions on infrastructure and information technology levels for this problem; however, to ensure persistent global status, we need a solution on an application level. The solution would be:

- for values persisted in the database, assign ($T \leftarrow 1, I \leftarrow 0, F \leftarrow 0$);
- for values in cache, assign ($T \leftarrow 0, I \leftarrow 1, F \leftarrow 0$); and
- for values that are neither in cache nor persisted in the database (Default), assign ($T \leftarrow 0, I \leftarrow 0, F \leftarrow 1$).

Exchanging messages between Microservices right now would be different; that represents the Microservice state. Listing 2.2 presents the enhanced neutrosophic-based e-Commerce cart. The status represents the neutrosophic-based proposed enhancement, where book status means that this purchase has been persisted, while chapter status means that it is still in the cache. Fig. 2.7 presents the enhanced neutrosophic-based Microservice state representation.

Listing 2.2 Neutrosophic-based e-Commerce in JSON format

```
{
  "guid" : "abc123xyz",
  "description" : "Purchase Operation Pending",
  "cartItems" : [
    {
      "book":"Microservices and DSLs",
      "status": "(1,0,0)"
    },
    {
      "chapter":"Enterprise Integration",
      "status": "(0,1,0)"
    }
  ],
  "nodeID" : "1234"
}
```

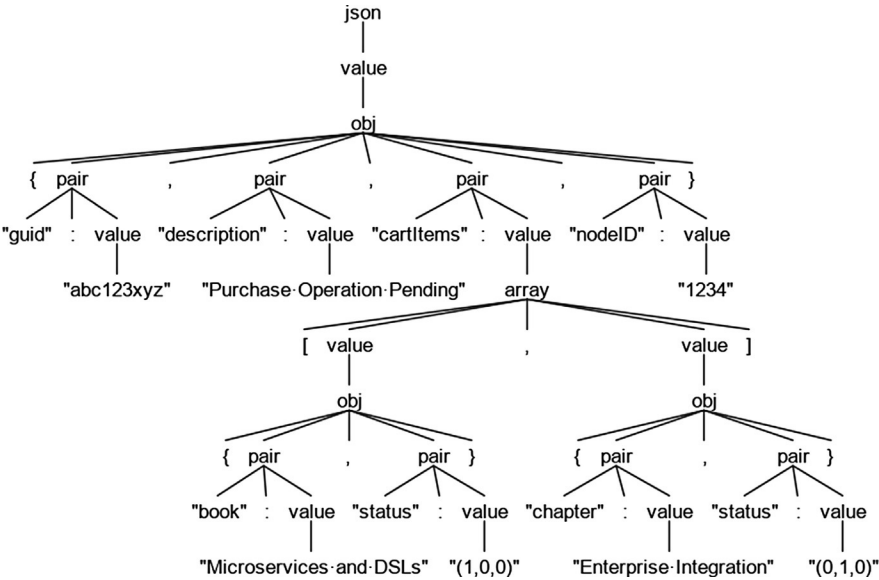


Fig. 2.7 Neutrosophic DSL parse tree.

2.8.3 Neutrosophic rules engine

The main advantage of a Rules Engine is the separation of rules from the programming language, and the application. Using a Rules Engine generally gives business developers the ability to focus on technical domains, and business experts the ability to

focus on the business rules. In this chapter, JSON will also be utilized to represent Rules in Rules Engines. To continue the case study, the focus will be on business rules. Unified Service-Oriented Rules Language was presented in Yin et al. [17]. In this chapter, we will utilize only the rules, and reformat them in JSON format instead of XML. This means that Microservices have to exchange two types of JSON messages: Business Rules and Microservice state.

2.8.3.1 Business rules engine language design

Four types of tags are presented in Yin et al. [17]:

1. Logical tag set
 - (a) ruleset
 - (b) rule
 - (c) when
 - (d) then
 - (e) param
2. Service tag set
 - (a) service
3. Operator tag set
 - (a) opt
 - (b) and
 - (c) or
 - (d) not
 - (e) equal
 - (f) add
 - (g) sub
 - (h) multiply
 - (i) divide
 - (j) modulo
 - (k) addequal
 - (l) subequal
 - (m) multiplyequal
 - (n) divideequal
 - (o) moduloequal
 - (p) gt
 - (q) lt
 - (r) openparenthesi
 - (s) closeparenthesi
4. Environmental tag set
 - (a) package

2.8.3.2 Business rule example

Using the above-mentioned Rules Engine proposed modeling language, assume that business domain experts want to add the following business rule: Special Discount on Purchases if Delivery Country is Egypt, Items greater than 3, and Today is User Birthday. Listing 2.3 presents the specification of the above business rule in JSON

format. This specification will be the JSON message exchanged between different Microservices. Fig. 2.8 presents the parse tree of the same business rule.

2.8.3.3 ANTLR

Another Tool For Language Recognition (ANTLR) is a parser generator that uses LL(*) for parsing. ANTLR takes as input a grammar that specifies a language and generates as output source code for a recognizer for that language. ANTLR supports

Listing 2.3 Example of business rule in JSON format

```
{ "guid" : "BR000001 ",
  "Title" : "Business Rule 001",
  "Date" : "23/06/2019",
  "description" : "Special Discount on Purchases if Delivery
                  Country is Egypt, Items greater than 3, and Today is
                  User Birthday",
  "rule" : [
    {
      "Today" :
      {
        "equal" : "User Birthday"
      }
    },
    {
      "and" :
      {
        "items in cart" :
        {
          "gt" : "3"
        }
      }
    },
    {
      "and" :
      {
        "Delivery" :
        {
          "equal" : "Egypt"
        }
      }
    }
  ]
}
```

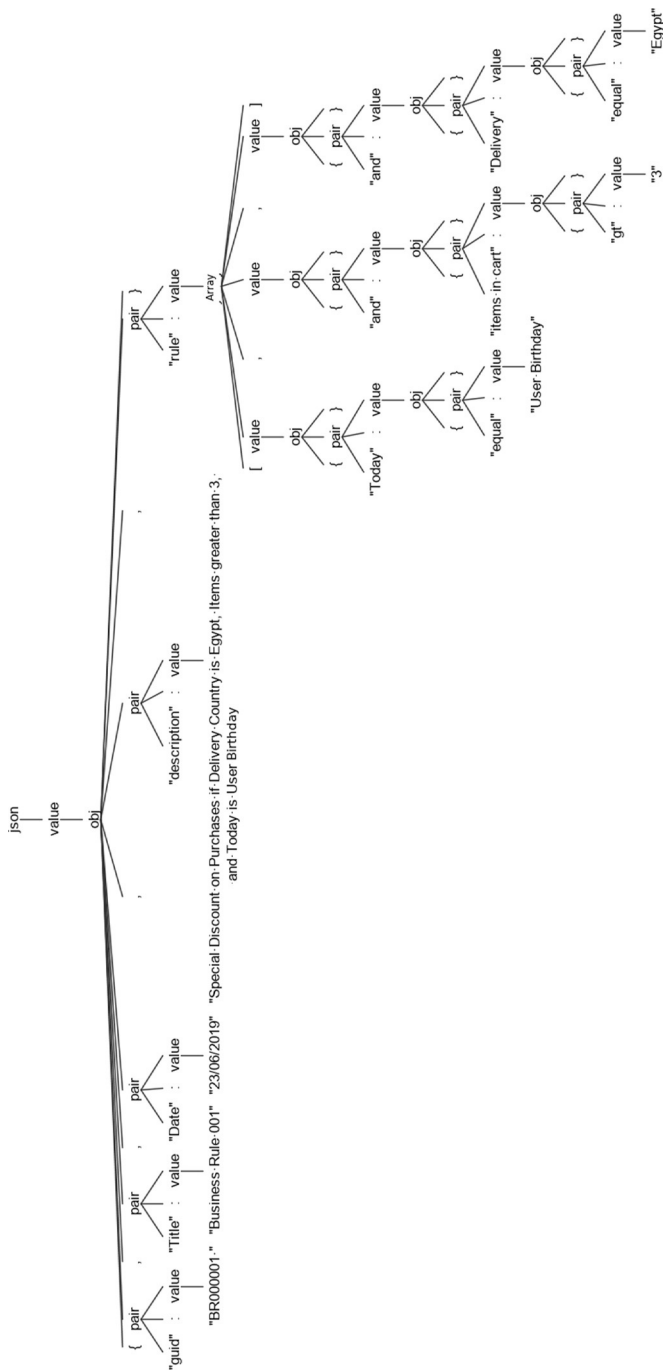


Fig. 2.8 Business rule parse tree.

generating code in Java, C#, JavaScript, Python2, and Python3. Language applications read sentences and react appropriately to the phrases and tokens (input symbols). A language is just a set of valid sentences. To react appropriately, an interpreter or translator has to recognize the input. The parser feeds off of tokens from the lexer, which feeds off of a char stream, to check syntax (language membership). ANTLR builds a parse tree while parsing. An ANTLR sequence of actions is presented in Eq. (2.5). ANTLR autogenerates listeners and visitors. For more information on ANTLR, the reader is referred to Parr [16]. The proposed Neutrosophic Rules Engine utilizes ANTLR as follows:

1. Define JSON Grammar file, as listed in Listing 2.4.
2. Parse the JSON Grammar file with ANTLR.
3. The proposed model will focus on Python3. Listing 2.5 displays the generated JSON Listener methods.
4. Taking a closer look at the generated JSON Listener, it is clear that generated methods are built based on Depth-First Search Graph Traversing of the JSON Grammar.
5. Business developers need to implement the required methods. For example, Listing 2.6 presents part of the implementation of the above-mentioned business rule. The proposed neutrosophic-based DSL and Rules Engine utilizes the novel open-source neutrosophic package presented in El-Ghareeb [22].

$$\text{chars} \rightarrow \text{LEXER} \rightarrow \text{tokens} \rightarrow \text{PARSER} \rightarrow \text{parse tree} \quad (2.5)$$

2.8.3.4 Decentralized rules engine

Right now, proposed Microservices architecture has the capability to exchange Microservice state in JSON format, and the capability to exchange different business rules. That leaves one last important decision for the proposed architecture: Rules Engine architecture. There are two models to apply the Business Rules at Microservices architecture: Centralized and Decentralized. The Centralized Rules Engine solution is the easiest, where there will be a central Rules Engine server, where business domain experts design and apply the business rules. The Rules Engine server is responsible for distributing the business rules to the different Microservices, and it is also responsible for consensus agreements. However, any centralized solution is not accepted any more for the same reasons discussed earlier, including scalability, fault tolerance, availability, and many other benefits.

Decentralization is the de facto standard in Microservices architecture. Microservices act as peers, and they must achieve consensus. This is a real challenge, as different Microservices are not the same state. This is where neutrosophy can tackle the challenge. There are different group aggregate methods and calculations based on Neutrosophic Sets and Theory. For detailed illustration, discussion, and examples of the utilized Neutrosophic Analytic Hierarchy Process, the reader is referred to Refs. [21, 23].

For example, assume that there are different Microservices state messages presenting the cart of a certain user based on the presentation at Section 2.8.2. There must be methods for aggregating different SVNss. A single-valued neutrosophic soft

Listing 2.4 JSON grammar [16]

```

/** Source: "The Definitive ANTLR 4 Reference" by Terence Parr
    from http://json.org */

grammar JSON;

json: object
    | array
    ;

object
    : '{' pair (',' pair)* '}'
    | '{' '}' // empty object
    ;

pair: STRING ':' value ;

array
    : '[' value (',' value)* ']'
    | '[' ']' // empty array
    ;

value
    : STRING
    | NUMBER
    | object // recursion
    | array // recursion
    | 'true' // keywords
    | 'false'
    | 'null'
    ;

STRING : '"' (ESC | ~["\\])* '"';
fragment ESC : '\\' (["\\/bfnrt] | UNICODE) ;
fragment UNICODE : 'u' HEX HEX HEX HEX ;
fragment HEX : [0-9a-fA-F] ;
NUMBER
    : '-'? INT '.' [0-9]+ EXP? // 1.35, 1.35E-9, 0.3, -4.5
    | '-'? INT EXP // 1e10 -3e4
    | '-'? INT // -3, 45
    ;
fragment INT : '0' | [1-9] [0-9]* ; // no leading zeros
fragment EXP : [Ee] [+|-]? INT ; // \- since - means "range"
            inside [...]
WS : [ \t\n\r]+ -> skip ;

```

Listing 2.5 JSON parser-Automatically generated

```

# Generated from java-escape by ANTLR 4.4
from antlr4 import *

# This class defines a complete listener for a parse tree
produced by ↪ JSONParser.

class JSONListener(ParseTreeListener):

    # Enter a parse tree produced by JSONParser#array.
    def enterArray(self, ctx):
        pass

    # Exit a parse tree produced by JSONParser#array.
    def exitArray(self, ctx):
        pass

    # Enter a parse tree produced by JSONParser#json.
    def enterJson(self, ctx):
        pass

    # Exit a parse tree produced by JSONParser#json.
    def exitJson(self, ctx):
        pass

    # Enter a parse tree produced by JSONParser#value.
    def enterValue(self, ctx):
        pass

    # Exit a parse tree produced by JSONParser#value.
    def exitValue(self, ctx):
        pass

    # Enter a parse tree produced by JSONParser#pair.
    def enterPair(self, ctx):
        pass

    # Exit a parse tree produced by JSONParser#pair.
    def exitPair(self, ctx):
        pass

    # Enter a parse tree produced by JSONParser#object.
    def enterObject(self, ctx):
        pass

    # Exit a parse tree produced by JSONParser#object.
    def exitObject(self, ctx):
        pass

```

Listing 2.6 Business Rule 01-Subimplementation

```
# Business Rule 01 Implementation

import datetime
from JSONListener import JSONListener

class BusinessRule_One(JSONListener):
    self._no_of_items_in_cart = 0
    self._delivery_country = ""
    self._today = datetime.datetime.date()

    # Enter a parse tree produced by JSONParser#pair.
    def enterPair(self, ctx):
        if ctx == "items-in-cart":
            self._no_of_items_in_cart =
                ↪self.enterValue(ctx)

    # Enter a parse tree produced by JSONParser#value.
    def enterValue(self, ctx):
        return ctx
```

weighted arithmetic averaging (SVNSWA) operator and single-valued neutrosophic soft weighted geometric averaging (SVNSWGA) operator have been used to compare two single-valued neutrosophic soft numbers (SVNSNs) for aggregating different single-valued neutrosophic soft input arguments in a neutrosophic soft environment [24]. Both operators are implemented in El-Ghareeb [22].

$$SVNNWAA(z_1, z_2, \dots, z_n) = \sum_{j=1}^n w_j z_j = \left\langle 1 - \prod_{j=1}^n (1 - T_j)^{w_j}, \prod_{j=1}^n (U_j)^{w_j}, \prod_{j=1}^n (V_j)^{w_j} \right\rangle \quad (2.6)$$

$$SVNNWGA(z_1, z_2, \dots, z_n) = \prod_{j=1}^n z_j^{w_j} = \left\langle \prod_{j=1}^n (T_j)^{w_j}, 1 - \prod_{j=1}^n (1 - U_j)^{w_j}, 1 - \prod_{j=1}^n (1 - V_j)^{w_j} \right\rangle \quad (2.7)$$

2.9 Conclusion and future work

This chapter highlighted the importance to move from monolithic architectures to Microservices ones. Different Microservices setup architectures have been presented, highlighting the pros and cons of each one. We then moved forward with the most suitable Microservices architecture: The one that provides the most advantages to fulfill and satisfy Microservices architecture objectives. However, Microservices architectures are very complex and face some significant challenges. Some of those challenges are depicted through an e-Commerce case study. Those challenges can

be solved utilizing DSLs. The main contribution of this chapter is utilizing Microservices logs as DSL in an innovative manner to present rules engines capabilities to Microservices architecture, and to help it overcome architectural challenges, and provide flexibility and configurability.

References

- [1] M. Villamizar, O. Garcés, H. Castro, M. Verano, L. Salamanca, R. Casallas, S. Gil, Evaluating the monolithic and the Microservice architecture pattern to deploy web applications in the cloud, in: 2015 10th Computing Colombian Conference (10CCC), 2015, pp. 583–590. <https://doi.org/10.1109/ColumbianCC.2015.7333476>.
- [2] Cesardelatorre, .NET Microservices. Architecture for Containerized .NET Applications (Online), 2019, <https://docs.microsoft.com/en-us/dotnet/standard/microservices-architecture>. Accessed 14 February 2019.
- [3] J. Soldani, D.A. Tamburri, W.-J.V.D. Heuvel, The pains and gains of Microservices: a systematic grey literature review, *J. Syst. Softw.* 146 (2018) 215–232, <https://doi.org/10.1016/j.jss.2018.09.082>.
- [4] O. Zimmermann, Microservices tenets, *Comput. Sci. Res. Dev.* 32 (3) (2017) 301–310, <https://doi.org/10.1007/s00450-016-0337-0>.
- [5] T. Cerny, Aspect-oriented challenges in system integration with Microservices, SOA and IoT, *Enterp. Inf. Syst.* (2018) 1–23, <https://doi.org/10.1080/17517575.2018.1462406>.
- [6] T. Cerny, M.J. Donahoo, J. Pechanec, Disambiguation and comparison of SOA, Microservices and self-contained systems, in: *Proceedings of the 2017 Research in Adaptive and Convergent Systems, RACS 2017*, 2017, pp. 228–235.
- [7] G. Pardon, C. Pautasso, O. Zimmermann, Consistent disaster recovery for Microservices: the BAC theorem, *IEEE Cloud Comput.* 5 (1) (2018) 49–59, <https://doi.org/10.1109/MCC.2018.011791714>.
- [8] A. Furda, C. Fidge, O. Zimmermann, W. Kelly, A. Barros, Migrating enterprise legacy source code to Microservices: on multitenancy, statefulness, and data consistency, *IEEE Softw.* 35 (3) (2018) 63–72, <https://doi.org/10.1109/MS.2017.440134612>.
- [9] J. Stubbs, W. Moreira, R. Dooley, Distributed systems of Microservices using Docker and Serfnode, in: 2015 7th International Workshop on Science Gateways, 2015, pp. 34–39. <https://doi.org/10.1109/TWSG.2015.16>.
- [10] C. Anderson, Docker [Software engineering], *IEEE Softw.* 32 (3) (2015) 102–c3, <https://doi.org/10.1109/MS.2015.62>.
- [11] D. Jaramillo, D.V. Nguyen, R. Smart, Leveraging Microservices architecture by using Docker technology, in: *SoutheastCon 2016*, 2016, pp. 1–5, <https://doi.org/10.1109/SECON.2016.7506647>.
- [12] N. Naik, Building a virtual system of systems using Docker swarm in multiple clouds, in: 2016 IEEE International Symposium on Systems Engineering (ISSE), 2016, pp. 1–3, <https://doi.org/10.1109/SysEng.2016.7753148>.
- [13] D.R. Cacciagrano, R. Culmone, IRON: reliable domain specific language for programming IoT devices, *Internet Things* (2018), <https://doi.org/10.1016/j.iot.2018.09.006>.
- [14] I. Dejanović, R. Vadera, G. Milosavljević, Ž. Vuković, TextX: a Python tool for domain-specific languages implementation, *Knowl. Based Syst.* 115 (2017) 1–4, <https://doi.org/10.1016/j.knosys.2016.10.023>.
- [15] D. Kramer, T. Clark, S. Oussena, MobDSL: a domain specific language for multiple mobile platform deployment, in: 2010 IEEE International Conference on Networked Embedded Systems for Enterprise Applications (NESEA), IEEE, 2010, pp. 1–7.

- [16] T. Parr, *The Definitive ANTLR 4 Reference*, Pragmatic Bookshelf, USA, 2013.
- [17] Z. Yin, C. Wu, S. Li, X. Li, T. Liu, A service-oriented business rules designer based on rule engine, in: S. Sambath, E. Zhu (Eds.), *Frontiers in Computer Education*, Springer, Berlin, Heidelberg, 2012, pp. 675–682, https://doi.org/10.1007/978-3-642-27552-4_90.
- [18] P. de Leusse, B. Kwolek, K. Zieliński, A middleware infrastructure for multi-rule-engine distributed systems, in: E. Di Nitto, R. Yahyapour (Eds.), *Towards a Service-Based Internet*, Springer, Berlin, Heidelberg, 2010, pp. 231–232.
- [19] M. Proctor, Drools: a rule engine for complex event processing, in: A. Schürr, D. Varró, G. Varró (Eds.), *Applications of Graph Transformations With Industrial Relevance*, Springer, Berlin, Heidelberg, 2012.
- [20] İ. Otay, C. Kahraman, A state-of-the-art review of neutrosophic sets and theory, in: C. Kahraman, İ. Otay (Eds.), *Fuzzy Multi-Criteria Decision-Making Using Neutrosophic Sets*, Springer International Publishing, Cham, 2019, pp. 3–24. https://doi.org/10.1007/978-3-030-00045-5_1.
- [21] N.A. Nabeeh, F. Smarandache, M. Abdel-Basset, H.A. El-Ghareeb, A. Aboelfetouh, An integrated neutrosophic-TOPSIS approach and its application to personnel selection: a new trend in brain processing and analysis, *IEEE Access* 7 (2019) 29734–29744, <https://doi.org/10.1109/ACCESS.2019.2899841>.
- [22] H.A. El-Ghareeb, Novel open source python neutrosophic package, *Neutrosophic Sets Syst.* 25 (2019) 136.
- [23] N.A. Nabeeh, M. Abdel-Basset, H.A. El-Ghareeb, A. Aboelfetouh, Neutrosophic multi-criteria decision making approach for IoT-based enterprises, *IEEE Access* 7 (2019) 59559–59574, <https://doi.org/10.1109/ACCESS.2019.2908919>.
- [24] C. Jana, M. Pal, A robust single-valued neutrosophic soft aggregation operators in multi-criteria decision making, *Symmetry* 11 (1) (2019), <https://doi.org/10.3390/sym11010110>.